

INSTITUTO FEDERAL FLUMINENSE
CAMPUS BOM JESUS DO ITABAPOANA
ENGENHARIA DE COMPUTAÇÃO
ALGORITMOS E TÉCNICAS DE PROGRAMAÇÃO

PARADIGMAS DE LINGUAGEM À PROGRAMAÇÃO

**CARLOS HENRIQUE BRASIL TEIXEIRA DE OLIVEIRA
MÁRCIO ALVES TEIXEIRA JÚNIOR
MIGUEL ARCANJO BORTOLINI PIMENTEL
PAOLA GUALANDE RIBEIRO BOECHAT
PEDRO HENRIQUE ROCHA DE ANDRADE**

Bom Jesus do Itabapoana, RJ
2024

Links Úteis:

Slide:

https://www.canva.com/design/DAF_Jvpz9Qc/AwqWxxhfpb-rkuY-0c93ZA/edit

Slides Roteiro:

- Introdução
 - Definição
 - História.
- Tipos
 - Programação Imperativa (Paola):
 - Programação Lógica (Paola):
 - Programação Orientado a Objetos (Carlos):
 - Programação Procedural (Carlos):
 - Programação Funcional (Márcio):
 - Programação Declarativa - Banco de Dados (Márcio):
 - Programação Orientada à Eventos (Miguel)
 - Programação Concorrente (Miguel)
- Exemplos Mundo Real
 - Aplicações Baseadas em Paradigmas e quais paradigmas foram utilizados (Op)
- Conclusão
 - Finalização e revisão de conteúdo (Só recapitular)

Referencial Teórico:

https://cm-kls-content.s3.amazonaws.com/201802/INTERATIVAS_2_0/ALGORITMO_S_E_TECNICAS_DE_PROGRAMACAO/U1/LIVRO_UNICO.pdf

<https://guia.dev/pt/pillars/languages-and-tools/programming-paradigms.html>

<http://www.decom.ufop.br/romildo/2016-2/bcc222/slides/01-introducao.pdf>

https://pt.wikipedia.org/wiki/Paradigma_de_programa%C3%A7%C3%A3o

[https://github.com/filipanselmo11/Lisp/blob/master/Conceitos%20de%20Linguagem%20de%20Programa%C3%A7%C3%A3o%20\(9a%20Edi%C3%A7%C3%A3o\)%20-%20Robert%20W.%20Sebesta%20-%20trabalho.pdf](https://github.com/filipanselmo11/Lisp/blob/master/Conceitos%20de%20Linguagem%20de%20Programa%C3%A7%C3%A3o%20(9a%20Edi%C3%A7%C3%A3o)%20-%20Robert%20W.%20Sebesta%20-%20trabalho.pdf)

Link para pasta de Apostilas Técnicas:  Programação

ChatGPT: <https://chat.openai.com/share/36ba202c-f485-4154-b7ac-282db596453c>

Sumário

[PARADIGMAS DE LINGUAGEM À PROGRAMAÇÃO](#)

[Links Úteis:](#)

[Definição](#)

[História](#)

[Tipos](#)

[Programação Imperativa \(Paola\):](#)

[Programação Lógica \(Paola\):](#)

[Programação Orientado a Objetos \(Carlos\):](#)

[Programação Procedural \(Carlos\):](#)

[Programação Funcional \(Márcio\):](#)

[Programação Declarativa - Banco de Dados \(Márcio\):](#)

[Programação Orientada à Eventos \(Miguel\):](#)

[Programação Concorrente \(Miguel\):](#)

[Exemplos do Mundo Real](#)

[Conclusão](#)

[Referências Bibliográficas](#)

Definição

Em curtas palavras, paradigma pode ser associado com um modelo padrão de execução. Existem diversos paradigmas, alguns antigos que já estão em desuso no mercado, e outros que estão quase presente em todos os projetos, como orientação à objetos que é um paradigma que vamos falar mais pra frente.

Basicamente um paradigma dita como será a execução e a estrutura daquele programa. Existem diversas linguagens de programação, como as citadas anteriormente pelo outro grupo, e, as diferentes linguagens, usam diferentes conjuntos de instruções (paradigmas). Além disso, existem algumas que foram criadas para suportar apenas um paradigma específico como o Haskell e o Funcional, enquanto outras como C++, Python suportam múltiplos paradigmas.

História

Surgiram nos primórdios da computação e evoluíram ao longo do tempo em resposta às demandas por maior expressividade, abstração e eficiência. O paradigma imperativo, por exemplo, foi um dos primeiros a ser utilizado, seguindo uma sequência de instruções para controlar o fluxo do programa. Com o advento do paradigma orientado a objetos, introduzido na década de 1960 com o desenvolvimento da linguagem Simula, surgiu a ideia de encapsulamento, herança e polimorfismo. Já o paradigma funcional, influenciado pela teoria dos lambda-cálculos, promoveu uma abordagem mais declarativa e matemática na resolução de problemas. Ao longo dos anos, novos paradigmas como a programação lógica, concorrente e baseada em eventos foram desenvolvidos, ampliando ainda mais o conjunto de ferramentas disponíveis para os desenvolvedores. A compreensão da história dos paradigmas de programação nos ajuda a contextualizar sua evolução e entender como diferentes abordagens foram influenciadas pelo avanço da tecnologia e pelas necessidades dos desenvolvedores.

Tipos

Para melhor compreensão, vamos exemplificar alguns dos tipos de paradigmas de linguagens de programação. Existem muitos, então não dá pra exibir todos, mas listamos os mais utilizados. Vale ressaltar que os paradigmas como dito anteriormente são comuns a várias linguagens, como Python, C, C++, Java, Javascript e outros, os exemplos a seguir estão descritos em Python.

Programação Imperativa (Paola):

Estrutura em um conjunto de sequência de instruções que alteram o estado do programa:

```
# Exemplo de programação imperativa em Python
def calcular_media(lista):
    soma = 0
    for elemento in lista:
        soma += elemento
    media = soma / len(lista)
    return media

notas = [8, 7, 9, 6, 8]
media_notas = calcular_media(notas)
print("A média das notas é:", media_notas)
```

Programação Lógica (Paola):

Estrutura com base nas regras de lógica e inferências (pouco utilizada, mais pra manipulações)

```
# Exemplo de programação lógica em Python com PyDatalog
from pyDatalog import pyDatalog

pyDatalog.create_terms('X, Y')
```

```
# Regras lógicas
+ (X == 2)
Y == X * 3

print("O valor de Y é:", Y)
# resultado: Y = 6
```

Programação Orientado a Objetos (Carlos):

O programa é estruturado em torno de objetos que contêm campos (atributos) de dados e o código na forma de procedimentos, com métodos.

```
# Exemplo de programação orientada a objetos em Python
```

```
class Pessoa:
```

```
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

```
    def mostrar_informacoes(self):
        print("Nome:", self.nome)
        print("Idade:", self.idade)
```

```
# Criando um objeto Pessoa
pessoa1 = Pessoa("João", 30)
pessoa1.mostrar_informacoes()
```

Programação Procedural (Carlos):

Programa estrutura para procedimentos de rotina, que contém um conjunto de instruções genéricos que servem para aquele tipo de necessidade.

```
# Exemplo de programação procedural em Python
def calcular_media(lista):
```

```
soma = 0
for elemento in lista:
    soma += elemento
media = soma / len(lista)
print("A média das notas é:", media)
```

```
notas = [8, 7, 9, 6, 8]
calcular_media(notas)
```

Programação Funcional (Márcio):

O programa é estruturado por meio de várias funções puras e imutáveis, funções genéricas para aproveitar códigos e não ter de ficar escrevendo várias vezes a mesma coisa:

```
# Exemplo de programação funcional em Python
def dobrar_elementos(lista):
    return list(map(lambda x: x * 2, lista))

numeros = [1, 2, 3, 4, 5]
dobro = dobrar_elementos(numeros)
print("O dobro dos números é:", dobro)
```

Programação Declarativa - Banco de Dados (Márcio):

O programa descreve o que deve ser feito, muito comum ser utilizado em Sistemas Gerenciadores de Banco de Dados (SGBD), SQL:

```
-- Exemplo de programação declarativa em SQL
//selecione os atributos/campos
SELECT nome, idade
```

```
//na coleção/tabela
FROM clientes
//para todos os cadastros que cidade = rio
WHERE cidade = 'Rio de Janeiro';
```

Programação Orientada à Eventos (Miguel):

o Programa é estruturado para atender a ocorrência de eventos, por exemplo para aquelas aplicações que são sempre atualizadas (Bolsa de Valores, Tráfego de Rede):

```
# Exemplo de programação orientada a eventos em Python com Tkinter
import tkinter as tk

def clique_botao():
    label.config(text="Botão clicado!")

root = tk.Tk()
root.geometry("200x100")

botao = tk.Button(root, text="Clique aqui", command=clique_botao)
botao.pack()

label = tk.Label(root, text="")
label.pack()

root.mainloop()
```

Programação Concorrente (Miguel):

Programa é estruturado para múltiplas tarefas serem executadas ao mesmo tempo, exemplo uma aplicação que exibe mais de um item na tela ao mesmo tempo:


```
# Exemplo de programação concorrente em Python com threading
import threading
import time

def imprimir_numeros():
    for i in range(1, 6):
        print("Número:", i)
        time.sleep(1)

def imprimir_letras():
    for letra in 'ABCDE':
        print("Letra:", letra)
        time.sleep(1)

# Criando duas threads
thread1 = threading.Thread(target=imprimir_numeros)
thread2 = threading.Thread(target=imprimir_letras)

# Iniciando as threads
thread1.start()
thread2.start()

# Aguardando as threads terminarem
thread1.join()
thread2.join()
```

Exemplos do Mundo Real

Esses paradigmas são utilizados por programadores profissionais o tempo todo, em qualquer lugar do mundo e muita das vezes, nas mais diversas linguagens. Existem alguns que são próprios para utilizar em algum projeto, outros que se encaixam em vários, é mais a critério do programador, mas aqui vão alguns exemplos de uso:

Programação Imperativa:

Um exemplo comum de programação imperativa é o desenvolvimento de sistemas de controle de estoque em empresas. Nesse caso, os programadores utilizam instruções diretas para manipular os dados do estoque, como adicionar ou remover itens, atualizar quantidades e calcular o valor total.

Programação Orientada a Objetos:

Um exemplo de aplicação da programação orientada a objetos pode ser encontrado em sistemas de gestão de bibliotecas. Os livros, os usuários e as transações de empréstimos são modelados como objetos com atributos e comportamentos específicos, facilitando a organização e manutenção do sistema.

Programação Funcional:

Um exemplo de uso da programação funcional está presente em sistemas de processamento de dados financeiros. Nesse caso, as funções são tratadas como cidadãos de primeira classe, permitindo a aplicação de operações de transformação e filtragem de dados de forma eficiente e concisa.

Programação Declarativa - Banco de Dados:

A programação declarativa é comumente utilizada em sistemas de gerenciamento de banco de dados, onde consultas SQL são usadas para recuperar e manipular dados. Em vez de especificar passo a passo como os dados devem ser acessados, os desenvolvedores declaram as condições que os dados devem satisfazer.

Programação Orientada a Eventos:

Em sistemas de monitoramento de tráfego de rede, a programação orientada a eventos é frequentemente empregada. Nesse contexto, eventos como a chegada de pacotes de dados, a conexão de novos dispositivos e a detecção de ameaças são tratados de forma assíncrona, permitindo uma resposta rápida e eficaz.

Programação Concorrente:

Em sistemas de processamento de transações financeiras em tempo real, a programação concorrente desempenha um papel crucial. Múltiplas transações

podem ser processadas simultaneamente, garantindo uma resposta rápida e evitando atrasos no processamento.

Conclusão

Ao explorarmos os diferentes paradigmas de programação e seus exemplos de aplicação no mundo real, fica evidente a diversidade de abordagens disponíveis para os desenvolvedores. Cada paradigma oferece suas próprias vantagens e desafios, e a escolha do paradigma adequado depende das necessidades do projeto e das preferências dos desenvolvedores. No entanto, é importante lembrar que não existe um paradigma "melhor" ou "pior" - cada um tem seu lugar e sua utilidade no desenvolvimento de software. Portanto, os desenvolvedores devem estar familiarizados com uma variedade de paradigmas e saber quando e como aplicá-los de forma eficaz. Em última análise, a compreensão dos paradigmas de programação é essencial para se tornar um programador mais versátil e eficiente, capaz de enfrentar os desafios do desenvolvimento de software de forma criativa e inovadora.

Referências Bibliográficas

Paradigmas de Programação. Disponível em:
<<https://guia.dev/pt/pillars/languages-and-tools/programming-paradigms.html>>.

Acesso em: 10 mar. 2024.

SEBESTA, R. W. **Conceitos de Linguagens de Programação - 11.ed.** [s.l.] Bookman Editora, 2018.

TUCKER, A.; NOONAN, R. **Linguagens de Programação - 2.ed.: Princípios e Paradigmas.** [s.l.: s.n.].

MALAQUIAS, J. R. Paradigmas de Programação. [s.d.].

Algoritmos e técnicas de programação. [s.l.] Editora e Distribuidora Educacional S.A., 2021.

Referências Bibliográficas:

[1] SEBESTA, Robert W. Conceitos de Linguagens de Programação - 11.ed. [S.l.: s.n.], 2018. Disponível em:

[https://github.com/filipanselmo11/Lisp/blob/master/Conceitos%20de%20Linguagem%20de%20Programa%C3%A7%C3%A3o%20\(9a%20Edi%C3%A7%C3%A3o\)%20-%20Robert%20W.%20Sebesta%20-%20trabalho.pdf](https://github.com/filipanselmo11/Lisp/blob/master/Conceitos%20de%20Linguagem%20de%20Programa%C3%A7%C3%A3o%20(9a%20Edi%C3%A7%C3%A3o)%20-%20Robert%20W.%20Sebesta%20-%20trabalho.pdf). Acesso em: 10 mar. 2024.

[2] Paradigmas de Programação. Disponível em:

<https://guia.dev/pt/pillars/languages-and-tools/programming-paradigms.html>. Acesso em: 10 mar. 2024.

[3] UFOP - Universidade Federal de Ouro Preto. Departamento de Computação. ROMILDO, Carlos. Introdução à Programação. [S.l.: s.n.], 2016. Disponível em:

<http://www.decom.ufop.br/romildo/2016-2/bcc222/slides/01-introducao.pdf>. Acesso em: 10 mar. 2024.

[4] WIKIPÉDIA. Paradigma de programação. Disponível em:

https://pt.wikipedia.org/wiki/Paradigma_de_programa%C3%A7%C3%A3o. Acesso em: 10 mar. 2024.

[5] LIVRO ÚNICO. Algoritmos e Técnicas de Programação. Disponível em:

https://cm-kls-content.s3.amazonaws.com/201802/INTERATIVAS_2_0/ALGORITMO_S_E_TECNICAS_DE_PROGRAMACAO/U1/LIVRO_UNICO.pdf. Acesso em: 10 mar. 2024.